

Razvoj web aplikacija - Predavanje 10

Admin tablice, filteri i permission-aware UI

mag.ing. Josip Torić (SIT UNIZD)

14.12.2026.



Studij informacijske tehnologije
Sveučilišta u Zadru

Plan predavanja

- 1 Admin CRUD ekrani za natjecanja
- 2 Tablice s filterima i paginacijom
- 3 Modali za destruktivne akcije
- 4 CSV export
- 5 Permission-aware UI: faze i role u frontendu
- 6 Commitovi za ovo predavanje

Na kraju predavanja: admin može upravljati natjecanjima i pregledavati sve registracije s filterima i exportom.

Nakon ovog predavanja student može:

- implementirati admin CRUD ekrane s tablicom i formom
- dodati filtriranje i paginaciju na tablicu
- koristiti modalni dijalog za potvrdu destruktivnih akcija
- generirati CSV export na frontendu iz podataka s API-ja
- prilagoditi UI ovisno o korisnikovoj roli i fazi natjecanja

Admin panel: odgovornosti

Admin korisnik ima širi pristup od običnog korisnika. U našem sustavu:

Akcija	Samo admin
Kreiranje natjecanja	Da - klubovi ne mogu kreirati natjecanja
Pregled svih registracija	Da - klubovi vide samo svoje
Reset lozinke kluba	Da - klubovi si ne mogu sami resetirati
Kreiranje novih klubova	Da - samo admin onboarda klubove
CSV export	Da - admin treba podatke za organizaciju

Admin viewovi i club viewovi dijele isti API, ali imaju različite UI ekrane. Backend kontrolira pristup, frontend kontrolira prikaz.

Princip najmanjeg privilegija na frontendu

Ne prikazuj korisniku akcije koje ne smije izvršiti:

- Club korisnik ne vidi „Novo natjecanje” gumb jer ne može kreirati natjecanja
- Club ne vidi admin navigaciju, ali admin vidi sve
- Gumbi za akcije koje ovise o fazi se skrivaju ili onemogućuju

Ali: nikad se ne oslanjajte samo na frontend za sigurnost. Korisnik može:

- Otvoriti DevTools i promijeniti DOM
- Direktno slati HTTP requestove (curl, Postman)
- Modificirati JavaScript u browseru

Frontend sakrivanje je UX, backend provjera je sigurnost. Oboje su potrebni.

Admin vs Club viewovi

Admin vidi

- Sve klubove i njihove natjecatelje
- CRUD za natjecanja
- Sve registracije na natjecanje
- Mogućnost exporta podataka

Club vidi

- Samo svoj klub i svoje natjecatelje
- Listu natjecanja (read-only)
- Samo svoje registracije
- Akcije ovisno o fazi

Isti API endpointi, različiti viewovi. Backend kontrolira što korisnik **smije** - frontend kontrolira što korisnik **vidi**.

Tablica kao UI pattern

Tablica je najčešći način prikaza strukturiranih podataka u admin panelu. Dobra tablica ima:

Element	Svrha
Zaglavlje (header)	Nazivi stupaca, sortiranje klikom na header
Redovi	Jedan red = jedan entitet (natjecanje, registracija)
Akcije	Gumbi/linkovi u zadnjem stupcu (Uredi, Obriši, Detalji)
Filteri	Search bar i dropdown iznad tablice
Paginacija	Navigacija kroz stranice ispod tablice
Empty state	Poruka kad nema podataka koji zadovoljavaju filtere

Sortiranje na frontendu

Za male datasetove, sortiranje radimo u JavaScriptu:

- `Array.sort()` sortira in-place - koristite `[...array].sort()` za novi array
- Sortiranje stringova: `a.name.localeCompare(b.name)` za ispravno sortiranje dijakritika
- Sortiranje datuma: usporedba timestamp vrijednosti
- Multi-column sort: primarni i sekundarni kriterij

UX konvencija: klik na header stupca sortira po tom stupcu. Strelica pokazuje smjer (asc/desc). Drugi klik invertira smjer. Koristite `computed` - ne mutirajte originalni array.

Filtriranje: dizajn pattern

Filteri se kombiniraju - korisnik može istovremeno pretraživati i filtrirati po statusu:

- 1 Svaki filter je `ref()` - `search`, `statusFilter`, `categoryFilter`
- 2 `computed chain` primjenjuje sve filtere redom na originalni array
- 3 Paginacija se primjenjuje na filtrirani rezultat (ne na originalni)
- 4 Promjena filtera resetira stranicu na 1

svi podaci → search filter → status filter → sort → paginate
→ prikaz

Svaki korak je `computed` koji ovisi o prethodnom. Vue automatski recalculira kad se bilo koji input promijeni.

Admin tablica natjecanja

```
<script setup lang="ts">
import { ref, onMounted } from 'vue'
import api from '@api/client'

const competitions = ref([])
const loading = ref(true)

onMounted(async () => {
  try {
    const { data } = await api.get('/competitions')
    competitions.value = data
  } finally {
    loading.value = false
  }
})
</script>

<template>
<h1>Natjecanja</h1>
<RouterLink to="/admin/competitions/new" class="btn">
  Novo natjecanje
</RouterLink>
<div v-if="loading">Ucitavanje...</div>
<table v-else>
  <thead>
    <tr>
      <th>Naziv</th><th>Datum</th>
      <th>Lokacija</th><th>Faza</th><th>Akcije</th>
    </tr>
  </thead>
  <tbody>
    <tr v-for="c in competitions" :key="c.id">
      <td>{{ c.name }}</td>
      <td>{{ c.date }}</td>
      <td>{{ c.location }}</td>
      <td><StatusBadge :status="c.phase" /></td>
      <td>
        <RouterLink :to="/" admin/competitions/${c.id}">
          Detalji
        </RouterLink>
      </td>
    </tr>
  </tbody>
</table>
</template>
```

Filtriranje: search i status filter

```
<script setup lang="ts">
import { ref, computed } from 'vue'

const search = ref<string>('')
const statusFilter = ref<string>('all')

const filtered = computed(() => {
  return registrations.value.filter((r) => {
    const matchesSearch =
      r.lifter_name.toLowerCase()
        .includes(search.value.toLowerCase())
    const matchesStatus =
      statusFilter.value === 'all'
      || r.status === statusFilter.value
    return matchesSearch && matchesStatus
  })
})
</script>

<template>
  <input v-model="search" placeholder="Pretraži..." />
  <select v-model="statusFilter">
    <option value="all">Svi</option>
    <option value="active">Aktivni</option>
    <option value="withdrawn">Odjavljeni</option>
  </select>
  <!-- tablica koristi filtered umjesto registrations -->
</template>
```

computed se automatski recalculira kad se search ili statusFilter promijeni.

Paginacija na frontendu

```
<script setup lang="ts">
import { ref, computed } from 'vue'

const page = ref(1)
const perPage = 10

const totalPages = computed(() =>
  Math.ceil(filtered.value.length / perPage)
)

const paginated = computed(() => {
  const start = (page.value - 1) * perPage
  return filtered.value.slice(start, start + perPage)
})
</script>

<template>
  <!-- tablica koristi paginated -->
  <div class="pagination">
    <button @click="page--" :disabled="page <= 1">
      Prethodna
    </button>
    <span>{{ page }} / {{ totalPages }}</span>
    <button @click="page++" :disabled="page >= totalPages">
      Sljedeca
    </button>
  </div>
</template>
```

Paginacija: frontend vs backend

	Frontend paginacija	Backend paginacija
Kako radi	Dohvati sve, sliceaj u JS-u	Šalji <code>?page=2&per_page=10</code>
Kad koristiti	Mali dataset (<500 redova)	Veliki dataset
Prednost	Instant navigacija	Ne opterećuje memoriju
Mana	Spor inicijalni load	Svaka stranica = novi request

Za projekt s 20-50 registracija, frontend paginacija je sasvim dovoljna. Backend paginacija je implementirana na `/lifters` endpointu - frontend samo šalje query parametre.

Infinite scroll vs paginacija

Paginacija

- Korisnik eksplicitno traži sljedeću stranicu
- Zna koliko podataka ima (stranica 3/10)
- Može preskočiti na stranicu 7
- Standard za admin tablice

Infinite scroll

- Nove stavke se učitavaju automatski na scroll
- Korisnik ne zna koliko ima podataka
- Ne može preskočiti na sredinu
- Standard za feedove (Instagram, Twitter)

Za admin tablice koristimo paginaciju - admin treba pregled nad podacima i mogućnost skakanja na specifičnu stranicu. Infinite scroll je za beskonačne feedove poput društvenih mreža.

Responsivne tablice

HTML tablice na mobilnim ekranima postaju nečitljive kad imaju više od 3-4 stupca:

Pristup	Opis
Horizontalni scroll	<code>overflow-x: auto</code> na wrapper divu. Korisnik scrolla lijevo-desno.
Sakrivanje stupaca	Manje bitni stupci se skrivaju na manjim ekranima s <code>@media</code> .
Card layout	Ispod breakpointa, svaki red postaje kartica s labelom i vrijednošću.

Za studentski projekt, horizontalni scroll je dovoljno dobar i zahtijeva jednu liniju CSS-a. Card layout je produkcijski standard, ali zahtijeva više koda.

Modalni dijalog za potvrdu

```
<!-- src/components/ConfirmModal.vue -->
<script setup lang="ts">
defineProps<{
  show: boolean
  title: string
  message: string
}>()
defineEmits<{
  confirm: []
  cancel: []
}>()
</script>

<template>
  <div v-if="show" class="modal-overlay" @click="$emit('cancel')">
    <div class="modal" @click.stop>
      <h3>{{ title }}</h3>
      <p>{{ message }}</p>
      <div class="modal-actions">
        <button @click="$emit('cancel')">Odustani</button>
        <button @click="$emit('confirm')" class="btn-danger">
          Potvrdi
        </button>
      </div>
    </div>
  </div>
</template>
```

@click.stop sprečava propagaciju klika - klik na modal ne zatvara overlay.

Modali: UX smjernice

- **Kad koristiti modal:** destruktivne akcije (brisanje, odjava), potvrda akcije s posljedicama
- **Kad ne koristiti:** informativni sadržaj, forme za unos podataka (bolje zasebna stranica)
- **ESC zatvara modal** - korisnici to očekuju
- **Klik na overlay zatvara modal** - ali ne i klik unutar modala (@click.stop)
- **Focus trap** - Tab tipka ne smije izaći iz modala dok je otvoren

Modal prekida korisnikov tok rada - koristite ga samo kad je prekid opravdan (sprečavanje greške).

Destruktivne akcije: UX pattern

Odjava natjecatelja je nepovratna akcija. UX pattern za destruktivne akcije:

- 1 Gumb za akciju je **vizualno drugačiji** - crveni gumb umjesto plavog
- 2 Klik otvara **modal s pitanjem** - ne izvršava odmah
- 3 Modal jasno opisuje **posljedice** - „Odjava se ne može poništiti”
- 4 **Potvrđi** gumb je crveni, **Odustani** je neutralan
- 5 Opcionalno: zahtijevaj unos teksta za potvrdu (npr. GitHub brisanje repozitorija)

Alternativa modalu je **undo pattern**: akcija se izvrši odmah, ali korisnik ima 5 sekundi da klikne „Poništi”. Gmail koristi ovo za brisanje mailova.

Teleport u Vueu

Problem: modal je definiran unutar child komponente, ali treba biti renderiran na vrhu DOM stabla (izvan layouta, iznad svih elemenata).

- Bez Teleporta: modal je nested u parent DOM-u. CSS `z-index` i `overflow: hidden` na parentu mogu sakriti overlay.
- S Teleportom: modal se renderira izvan parent DOM-a, direktno u `<body>`.

`<Teleport to="body">` premješta sadržaj u `<body>` element. Komponenta ostaje u Vue stablu (props, events rade normalno), ali DOM node je na drugom mjestu. Ovo rješava `z-index` i `overflow` probleme.

Korištenje modala za odjavu

```
<script setup lang="ts">
import { ref } from 'vue'
import ConfirmModal from '@/components/ConfirmModal.vue'

const showModal = ref(false)
const selectedReg = ref(null)

function askWithdraw(reg) {
  selectedReg.value = reg
  showModal.value = true
}

async function confirmWithdraw() {
  await api.post(
    `/competitions/${compId}/registrations/${selectedReg.value.id}/withdraw`
  )
  showModal.value = false
  await refreshData()
}
</script>

<template>
<button @click="askWithdraw(reg)">Odjavi</button>
<ConfirmModal :show="showModal"
  title="Odjava natjecatelja"
  message="Jeste li sigurni? Odjava se ne može poništiti."
  @confirm="confirmWithdraw"
  @cancel="showModal = false" />
</template>
```

CSV export

```
// src/utils/csv.ts
export function downloadCSV(
  data: Record<string, unknown>[], filename: string
) {
  if (!data.length) return

  const headers = Object.keys(data[0])
  const rows = data.map((row) =>
    headers.map((h) => `${row[h] ?? ''}`).join(',')
  )

  const csv = [headers.join(','), ...rows].join('\n')
  const blob = new Blob([csv], { type: 'text/csv' })
  const url = URL.createObjectURL(blob)

  const a = document.createElement('a')
  a.href = url
  a.download = filename
  a.click()
  URL.revokeObjectURL(url)
}
```

Generiramo CSV na frontendu iz podataka koje već imamo. Za velike datasetove, backend bi mogao imati dedicerani export endpoint.

CSV export: korištenje u viewu

```
<script setup lang="ts">
import { downloadCSV } from '@/utils/csv'

function exportRegistrations() {
  const rows = registrations.value.map((r) => ({
    natjecatelj: r.lifter_name,
    kategorija: r.category,
    status: r.status,
    datum_prijave: r.registered_at,
  }))
  downloadCSV(rows, `registracije-${compId}.csv`)
}
</script>

<template>
  <button @click="exportRegistrations">
    Preuzmi CSV
  </button>
</template>
```

Permission-aware UI

Frontend prilagođava prikaz ovisno o **roli korisnika** i **fazi natjecanja**:

Element	Admin	Club (OPEN)	Club (CLOSED)
Gumb „Nova prijava”	vidljiv	vidljiv	skriven
Gumb „Odjavi”	vidljiv	vidljiv	skriven
Gumb „Uredi kategoriju”	vidljiv	vidljiv	skriven
Link „Novo natjecanje”	vidljiv	skriven	skriven
CSV export	vidljiv	skriven	skriven

Frontend sakriva gumbe za bolji UX, ali backend i dalje odbija nedozvoljene akcije. Nikad se ne oslanjajte samo na frontend za sigurnost.

Disabled vs Hidden

Kad korisnik nema pravo na akciju, imamo dva izbora:

Hidden (v-if)

- Element ne postoji u DOM-u
- Korisnik ne zna da opcija postoji
- Čišći UI
- Koristiti za role-based skrivanje

Disabled (:disabled)

- Element je vidljiv, ali neklikabilan
- Korisnik zna da opcija postoji
- Tooltip objašnjava zašto
- Koristiti za phase-based ograničenja

Club korisnik ne vidi „Novo natjecanje” jer tu akciju nikad neće smjeti izvesti. „Nova prijava” nakon roka je disabled jer ju je ranije mogao izvesti.

Faze natjecanja na frontendu

Backend šalje fazu natjecanja u API odgovoru. Frontend koristi tu informaciju za prilagodbu UI-ja:

Faza	Nova prijava	Promjena kat.	Odjava
OPEN	Omogućeno	Omogućeno	Omogućeno
PRELIM_PASSED	Disabled + tooltip	Omogućeno	Omogućeno
CLOSED	Hidden	Disabled	Disabled

Problem: korisnik može otvoriti stranicu u OPEN fazi, otići na kavu i pokušati akciju kad je već PRELIM_PASSED. Rješenja:

- Backend i dalje odbija - korisnik dobije grešku
- Periodički re-fetch podataka (polling)
- Prikazati countdown do roka na stranici

Bulk akcije u tablicama

Admin tablica s mnogo zapisa može trebati akcije nad više redova odjednom:

- Checkbox u svakom redu + „Select All” u headeru
- Toolbar s akcijama se pojavi kad je barem jedan red selektiran
- Akcije: „Exportaj odabrane”, „Obriši odabrane”
- Prikaz broja selektiranih: „5 od 23 odabrano”

Implementacija: `selectedIds = ref(new Set<number>())`. Toggle dodaje/uklanja ID. „Select All” dodaje sve ID-eve s trenutne stranice (ne sve stranice). Bulk akcija šalje listu ID-eva na backend.

Disabling akcija po fazi

```
<script setup lang="ts">
import { computed } from 'vue'

const canRegister = computed(() =>
  competition.value.phase === 'open'
)
const canModify = computed(() =>
  ['open', 'prelim_passed'].includes(competition.value.phase)
)
</script>

<template>
  <button @click="register" :disabled="!canRegister"
    :title="canRegister ? '' : 'Rok za prijave je istekao'">
    Nova prijava
  </button>
  <button @click="withdraw(reg)" v-if="canModify">
    Odjavi
  </button>
</template>
```

Koristimo `:disabled` umjesto `v-if` za gumbe koji su vidljivi ali nedostupni - korisnik vidi da opcija postoji, ali nije dozvoljena.

Role-based UI rendering

Tri pristupa za prikazivanje elemenata ovisno o roli:

Pristup	Primjer
Direktan v-if	<code>v-if="auth.isAdmin"</code> - jednostavno, ali ponovljeno
Helper funkcija	<code>v-if="canCreate('competition')"</code> - centralizirana logika
Zasebni viewovi	<code>AdminDashboard.vue</code> vs <code>ClubDashboard.vue</code>

Za naš projekt: admin i club imaju zasebne viewove jer je UI značajno drugačiji. Unutar zajedničkih komponenti koristimo `v-if="auth.isAdmin"` za manje razlike.

Audit trail: zašto pratiti akcije

U produkcijskom sustavu treba znati tko je što napravio i kad:

- Tko je odjavio natjecatelja? Admin ili klub?
- Kad je promijenjena kategorija? Prije ili nakon prelim roka?
- Je li admin resetirao lozinku kluba?

Razina	Kako
Backend logovi	Strukturirani logovi: user_id, akcija, timestamp, IP
Baza	created_at, updated_at, opcionalno updated_by
Frontend	Console.log za debug, ne za audit

Za ovaj projekt imamo registered_at na registraciji. U produkciji bismo dodali withdrawn_at, withdrawn_by i slično.

Commitovi za Predavanje 10

- `feat(web): add admin competitions CRUD screens`
- `feat(web): add admin registrations table with filters and pagination`
- `feat(web): add approve/reject UI with modal and reason`
- `feat(web): add CSV export button and download flow`
- `fix(web): disable actions based on competition phase (prelim/final/locked)`

Na satu: gradimo admin ekrane, testiramo filtriranje, gledamo kako se UI mijenja po fazama.

Što ide sljedeće (Predavanje 11)

- Production build i environment config
- Deploy na Railway ili Render
- CORS konfiguracija
- CI/CD pipeline
- Priprema za 2. obranu

Pitanja?